Kateryna Ivanova-Nekrasova, 26637

1. Quellcode

```
#include "stdio.h"
#include "stdlib.h"

#define INT "int"
#define CHAR "char"
#define SHORT "short"
#define GLOBAL "global int"
#define DYNAMIC "dynamic int"

using namespace std;

void printAdr(const char* arr, void* a, void* b);
void printAdrDist(const char* arr, void* a, void* b);
void printArrAdr(int arr[], int n);


int aG, bG;

int main()
{
    int aI, bI;
    short aS, bS;
    char aC, bC;

    char* dyn1 = (char*)malloc(sizeof(char));
    char* dyn2 = (char*)malloc(sizeof(char));

    register char reg;
    static char stat;


    int arr1[5];
    int* arr2 = (int*)malloc(sizeof(int)*5);


    //---------------------------- AUFGABE 1 ----------------------------

    printf("=== Global variables === \n__________________________\n");
    printAdr(GLOBAL, (void*)&aG, (void*)&bG);

    printf("========================\n"
        "=== Local variables ==="
        "\n__________________________\n");
    printAdr(INT, (void*)&aI, (void*)&bI);

    printAdr(SHORT, (void*)&aS, (void*)&bS);
    printAdr(CHAR, (void*)&aC, (void*)&bC);

    printf("\n========================\n"
        "=== Distance ==="
        "\n__________________________\n");
    printAdrDist(GLOBAL, (void*)&aG, (void*)&bG);
    printAdrDist(INT, (void*)&aI, (void*)&bI);
    printAdrDist(SHORT, (void*)&aS, (void*)&bS);
```

```
    printAdrDist(CHAR, (void*)&aC, (void*)&bC);

    //---------------------------- AUFGABE 2 ----------------------------
    printf("\n========================\n=== Dynamic variables ===
\n_________________________\n");
    printAdr(DYNAMIC, (void*)&dyn1, (void*)&dyn2);
    printAdrDist(DYNAMIC, (void*)&dyn1, (void*)&dyn2);

    //---------------------------- AUFGABE 3 ----------------------------
    printf("\n========================\n=== Static array ===
\n_________________________\n");
    printArrAdr(arr1, 5);
    printf("\n========================\n=== Dynamic array ===
\n_________________________\n");
    printArrAdr(arr2, 5);

}

void printAdr(const char *arr, void* a, void* b)
{
    printf("Adress of %s a = %X\n", arr, a);
    printf("Adress of %s b = %X\n\n", arr, b);
}

void printAdrDist(const char *arr, void* a, void* b)
{
    printf("Distance between %s a, b = %d\n\n", arr, abs((long)b - (long)a));
}

void printArrAdr(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        printf("Adress of the %d. element: %X\n", i + 1, (long)&arr[i]);
    }
}
```

2. Aufgabenstellungen

1)

```
=== Global variables ===
_________________________
Adress of global int a = 33A138
Adress of global int b = 33A13C

========================
=== Local variables ===
_________________________
Adress of int a = 6FFE58
Adress of int b = 6FFE4C

Adress of short a = 6FFE40
Adress of short b = 6FFE34

Adress of char a = 6FFE2B
Adress of char b = 6FFE1F


========================
=== Distance ===
_________________________
Distance between global int a, b = 4

Distance between int a, b = 12

Distance between short a, b = 12

Distance between char a, b = 12
```

- Globale Variablen werden in einem vom Compiler festgelegten festen Speicherbereich gespeichert und nebeneinander gestapelt, sodass der Abstand zwischen den Adressen zweier globaler Variablen ist gleich 4.
- Lokale Variablen befinden sich in ein Stack.
  der Stack ist immer 16-Byte ausgerichtet, was dazu führt, dass der Abstand zwischen Variablenadressen größer ist als die Größe des Datentyps (16-4=12)

(Sie benötigen 4 Bytes, da die Größe von Adressen (Zeigern) nur 4 Bytes beträgt)

2)

```
=== Dynamic variables ===
_________________________
Adress of dynamic int a = 6FFE10
Adress of dynamic int b = 6FFE04

Distance between dynamic int a, b = 12
```

a) Dynamische Variablen befinden sich in ein Heap.
Heap ist ein Datenstruktur, mittels welcher ist dynamische Speicherverwaltung realisiert wird.
Im Vergleich zur Funktionsweise des Stacks ist der Heap langsamer, da die Variablen im Speicher verteilt sind und nicht oben auf dem Stack. Auch ein unterschied zwischen Stack und Heap ist, dass Stack hat ein fester Wert in Arbeitspeicher für jedes Programm, und mit Heap kann man im Programm so viel Arbeitsspeicher benutzen, wie es zu verfügung steht.
Der Stack ist mehr sicherer, als heap, weil Heapüberlauf zu Überschreibung von Heap-Daten (Variablen und Zeiger), oder zu Beschädigung der internen Informationsstruktur des Heaps führen.

b) malloc ist eine Funktion, die einfach einen Speicherblock der als Parameter übergebenen Größe zuweist und einen void-Zeiger zurückgibt. Um die Adresse dieses Speichers einem Zeiger auf ein Objekt zuzuweisen, ist es notwendig zuerst den Rückgabewert der Funktion explizit in den gewünschten Typ umwandeln. In diesem Fall erfolgt keine Kontrolle auf erfolgreiche Speicherzuweisung.
Der Operator « new» hingegen allokiert Speicher mit einer Kontrolle (im Fehlerfall generiert er bad_alloc) und mit einem Konstruktoraufruf für das Objekt. Das heißt, Objekt sofort initialisiert werden kann. Außerdem kann man für Typen new, delete oder deren Versionen für Arrays überladen. Es ist möglich, ein Objekt an einer Adresse zu platzieren, an der Speicher vorab zugewiesen wurde, indem ein Zeiger als zweiter Parameter übergeben wird. In diesem Fall muss der Destruktor explizit aufgerufen werden, weil Wir weisen dem Objekt keinen Speicher zu, sondern übergeben nur einen Zeiger.

c) Spezifikator „register“ bewirkt, dass der Compiler versucht, diese Variable in einem CPU-Register zu speichern, anstatt sie nur im Speicher zuzuweisen. In diesem Fall funktionierten alle

Verweise auf die Variable außergewöhnlich schnell. Verwendung des Registerbezeichners nur eine Anfrage ist und möglicherweise nicht gewährt wird. Dem Compiler kann die register-Anrufe ignorieren. Der Grund dafür ist, dass nur eine begrenzte Anzahl von Variablen optimiert werden kann, um die Datenverarbeitung zu beschleunigen. Wenn diese Grenze überschritten wird, ignoriert der Compiler einfach weitere Register-"Anforderungen".

d) Speyifikator „static“ weist den Compiler an, die lokale Variable für die Lebensdauer des Programms beizubehalten, anstatt sie jedes Mal zu erstellen und zu zerstören, wenn sie in den Gültigkeitsbereich eintritt und ihn verlässt. Daher ermöglicht das Anheben lokaler Variablen in den Rang einer Statistik, dass ihre Werte zwischen Funktionsaufrufen beibehalten werden.

3)

```
=== Static array ===
_________________________
Adress of the 1. element: 6FFDDC
Adress of the 2. element: 6FFDE0
Adress of the 3. element: 6FFDE4
Adress of the 4. element: 6FFDE8
Adress of the 5. element: 6FFDEC

========================
=== Dynamic array ===
_________________________
Adress of the 1. element: 835D40
Adress of the 2. element: 835D44
Adress of the 3. element: 835D48
Adress of the 4. element: 835D4C
Adress of the 5. element: 835D50
```